

IFVG Sniper Model — Pine Script v6

TradingView indicator · defaults tuned for the 15-minute chart · 673 lines. Copy the code from the raw GitHub link, not from this PDF.

```
1 //@version=6
2 //
3 // IFVG Sniper Model (defaults tuned for the 15-minute chart)
4 // 1. Liquidity sweep — price wicks through a swing / previous-day high-low and closes back inside
5 // 2. V-shape recovery — fast displacement away from the sweep extreme (× ATR)
6 // 3. CISD — close through the open of the delivery series that made the extreme
7 // 4. IFVG — an opposing FVG is closed through by a displacement candle → sniper zone
8 // 5. Retest + sweep — price returns into the IFVG, sweeps minor liquidity and holds → entry
9 // 6. Targets — fixed 1R / 2R (default) with a draw on liquidity required beyond TP1
10 // Filters — higher-timeframe bias, London / New York killzones, max stop size,
11 // discount / premium entry, daily trade & loss limits, one position at a time
12 //
13 indicator("IFVG Sniper Model", "IFVG Sniper", overlay = true, max_bars_back = 500,
14 max_lines_count = 500, max_labels_count = 500, max_boxes_count = 500)
15
16 // ————— Inputs —————
17 const string G0 = "0 · High-Probability Filters"
18 useHtf = input.bool(true, "Trade with higher-timeframe bias", group = G0,
19 tooltip = "Longs only when the HTF close is above a rising EMA, shorts only below a falling EMA
20 (last closed HTF bar, no repaint).")
21 htfTf = input.timeframe("60", "HTF", group = G0, inline = "htf")
22 htfEmaLen = input.int(50, "EMA", minval = 1, group = G0, inline = "htf")
23 useKz = input.bool(true, "Entries only inside killzones", group = G0)
24 kz1 = input.session("0200-0500", "London", group = G0, inline = "kz")
25 kz2 = input.session("0700-1100", "New York", group = G0, inline = "kz")
26 kzTz = input.string("America/New_York", "Killzone timezone", group = G0)
27 dispBody = input.float(0.5, "Min body % of IFVG inversion candle", minval = 0, maxval = 1, step = 0.05,
28 group = G0,
29 tooltip = "The candle that closes through the FVG must be a displacement candle (body / range). 0
30 disables.")
31 pdMin = input.float(0.5, "Min retracement into the V leg", minval = 0, maxval = 1, step = 0.05, group
32 = G0,
33 tooltip = "Entry must sit at least this deep in the range from the sweep extreme to the V high
34 (bull) / low (bear). 0.5 = discount for longs, premium for shorts. 0 disables.")
35 maxDay = input.int(2, "Max trades per day", minval = 1, group = G0, inline = "day")
36 maxLossDay = input.int(1, "Stop after N losses", minval = 1, group = G0, inline = "day")
37 maxSlAtr = input.float(2.0, "Skip entry if stop > N × ATR", minval = 0.2, step = 0.1, group = G0)
38 minSlAtr = input.float(0.3, "Minimum stop size (× ATR)", minval = 0, step = 0.05, group = G0,
39 tooltip = "Stops tighter than this get widened so normal noise does not tag them.")
40
41 const string G1 = "1 · Liquidity Sweep"
42 swingLen = input.int(8, "Liquidity swing length", minval = 2, group = G1,
43 tooltip = "Bars on each side that define a swing high/low (a resting liquidity pool). Higher = only
44 major liquidity.")
45 usePdl = input.bool(true, "Add previous-day high / low as liquidity", group = G1)
46 useAsia = input.bool(true, "Add Asia session high / low as liquidity", group = G1, inline = "asia")
47 asiaSess = input.session("2000-0000", "", group = G1, inline = "asia")
48 maxPools = input.int(30, "Max pools tracked per side", minval = 5, maxval = 100, group = G1)
49 showPools = input.bool(false, "Show resting liquidity levels", group = G1)
50
51 const string G2 = "2 · V-Shape Recovery"
52 vMaxBars = input.int(6, "Max bars from extreme → recovery", minval = 1, group = G2)
53 vAtrMult = input.float(1.5, "Recovery displacement (× ATR)", minval = 0.1, step = 0.1, group = G2)
54 atrLen = input.int(14, "ATR length", minval = 1, group = G2)
55
56 const string G3 = "3 · CISD + IFVG"
57 alignBars = input.int(12, "Max bars sweep → CISD & IFVG", minval = 1, group = G3)
58 cisdScan = input.int(15, "CISD candle scan depth", minval = 1, maxval = 100, group = G3)
59 fvgLook = input.int(20, "FVG may form up to N bars before sweep", minval = 1, group = G3)
60 fvgMinAtr = input.float(0.1, "Min FVG size (× ATR)", minval = 0, step = 0.05, group = G3)
61
62 const string G4 = "4 · Retest + Second Sweep"
63 retestMax = input.int(16, "Max bars to wait for retest", minval = 1, group = G4)
64 minorLen = input.int(3, "Retest sweep lookback (bars)", minval = 1, group = G4,
65 tooltip = "On the retest, price must take the low (bull) / high (bear) of the previous N bars and
66 close back.")
67
68 const string G5 = "5 · Targets & Management"
```

```

62 tpMode      = input.string("Fixed R:R", "Target mode", options = ["Fixed R:R", "Liquidity (capped at
R:R)"], group = G5,
63     tooltip = "Fixed R:R: TP1 / TP2 exactly at the R multiples below.\nLiquidity (capped): nearest draw
on liquidity, never further than the R multiples.")
64 rr1         = input.float(1.0, "TP1 (R)", minval = 0.25, step = 0.25, group = G5, inline = "rr")
65 rr2         = input.float(2.0, "TP2 (R)", minval = 0.5, step = 0.25, group = G5, inline = "rr")
66 requireDol  = input.bool(true, "Require draw on liquidity beyond TP1", group = G5,
67     tooltip = "Only enter when an untouched liquidity pool sits at least TP1 away in the trade
direction.")
68 slBuf       = input.float(0.1, "SL buffer (× ATR)", minval = 0, step = 0.05, group = G5)
69 partialPct  = input.float(50, "Close % at TP1", minval = 0, maxval = 100, step = 5, group = G5)
70 beAfterTp1  = input.bool(true, "Move SL to entry after TP1", group = G5)
71 tradeMax    = input.int(96, "Close trade after N bars", minval = 1, group = G5)
72
73 const string G6 = "Display"
74 showBull    = input.bool(true, "Bullish setups", group = G6, inline = "d")
75 showBear    = input.bool(true, "Bearish setups", group = G6, inline = "d")
76 bullCol     = input.color(#089981, "Bull", group = G6, inline = "c")
77 bearCol     = input.color(#F23645, "Bear", group = G6, inline = "c")
78 tgtCol      = input.color(#26A69A, "Profit", group = G6, inline = "c")
79 slCol       = input.color(#EF5350, "Risk", group = G6, inline = "c")
80 showFailed  = input.bool(false, "Keep cancelled setups on chart (grey)", group = G6)
81 lblSizeIn   = input.string("Normal", "Label size", options = ["Small", "Normal", "Large"], group = G6)
82 showStats   = input.bool(true, "Show stats table", group = G6, inline = "t")
83 tablePos    = input.string("Top right", "", options = ["Top right", "Bottom right", "Top left", "Bottom
left"], group = G6, inline = "t")
84 string lblSz = lblSizeIn == "Large" ? size.large : lblSizeIn == "Normal" ? size.normal : size.small
85
86 // ----- Core series -----
87 atr          = ta.atr(atrLen)
88 minorLow     = ta.lowest(low, minorLen)[1]
89 minorHigh    = ta.highest(high, minorLen)[1]
90 ph          = ta.pivohigh(high, swingLen, swingLen)
91 pl          = ta.pivotlow(low, swingLen, swingLen)
92
93 // Higher-timeframe bias from the last CLOSED HTF bar (lookahead + [1] = no repaint)
94 htfData() =>
95     float e = ta.ema(close, htfEmaLen)
96     [close[1], e[1], e[2]]
97 [htfClose, htfEma, htfEmaPrev] = request.security(syminfo.tickerid, htfTf, htfData(), lookahead =
barmerge.lookahead_on)
98 bool bullBias = not useHtf or (htfClose > htfEma and htfEma > htfEmaPrev)
99 bool bearBias = not useHtf or (htfClose < htfEma and htfEma < htfEmaPrev)
100
101 // Killzones
102 bool inKz = not useKz or not na(time(timeframe.period, kz1, kzTz)) or not na(time(timeframe.period, kz2,
kzTz))
103
104 // Displacement candle (body share of range) for the IFVG inversion
105 float rng    = high - low
106 bool bullDisp = close > open and rng > 0 and (close - open) / rng >= dispBody
107 bool bearDisp = close < open and rng > 0 and (open - close) / rng >= dispBody
108
109 // ----- Liquidity pools -----
110 var array<float> bsLPx = array.new<float>() // buy-side liquidity (swing highs)
111 var array<int> bsLBar = array.new<int>()
112 var array<line> bsLLn = array.new<line>()
113 var array<float> ssLPx = array.new<float>() // sell-side liquidity (swing lows)
114 var array<int> sslBar = array.new<int>()
115 var array<line> sslLn = array.new<line>()
116
117 addPool(array<float> px, array<int> bx, array<line> ln, float p, int b) =>
118     line l = na
119     if showPools
120         l := line.new(b, p, bar_index, p, color = color.new(color.gray, 40), style = line.style_dotted,
extend = extend.right)
121     px.push(p)
122     bx.push(b)
123     ln.push(l)
124     if px.size() > maxPools
125         px.shift()
126         bx.shift()
127         line.delete(ln.shift())
128

```

```

129 // Removes every pool traded through this bar. Returns the most extreme pool that was
130 // swept AND reclaimed (wick through, close back) - d = 1 checks lows, d = -1 checks highs.
131 scanSweeps(array<float> px, array<int> bx, array<line> ln, int d) =>
132     float lvl = na
133     int lb = na
134     int n = px.size()
135     if n > 0
136         for i = n - 1 to 0
137             float p = px.get(i)
138             bool taken = d == 1 ? low < p : high > p
139             if taken
140                 bool reclaimed = d == 1 ? close > p : close < p
141                 if reclaimed and (na(lvl) or d * (lvl - p) > 0)
142                     lvl := p
143                     lb := bx.get(i)
144                     line l = ln.get(i)
145                     if not na(l)
146                         line.set_x2(l, bar_index)
147                         line.set_extend(l, extend.none)
148                     px.remove(i)
149                     bx.remove(i)
150                     ln.remove(i)
151     [lvl, lb]
152
153 // ----- Fair value gaps -----
154 var array<float> bearFvgTop = array.new<float>() // bearish FVGs → become bullish IFVGs
155 var array<float> bearFvgBot = array.new<float>()
156 var array<int> bearFvgBar = array.new<int>()
157 var array<float> bullFvgTop = array.new<float>() // bullish FVGs → become bearish IFVGs
158 var array<float> bullFvgBot = array.new<float>()
159 var array<int> bullFvgBar = array.new<int>()
160
161 addFvg(array<float> top, array<float> bot, array<int> bx, float t, float b, int x) =>
162     top.push(t)
163     bot.push(b)
164     bx.push(x)
165     if top.size() > 100
166         top.shift()
167         bot.shift()
168         bx.shift()
169
170 // Removes every FVG closed through this bar. Returns the most recent inverted FVG that
171 // formed at/after minBar. d = 1 → bearish FVG closed above, d = -1 → bullish FVG closed below.
172 scanInversions(array<float> top, array<float> bot, array<int> bx, int d, int minBar) =>
173     float zt = na
174     float zb = na
175     int zbar = na
176     int n = top.size()
177     if n > 0
178         for i = n - 1 to 0
179             bool inverted = d == 1 ? close > top.get(i) : close < bot.get(i)
180             if inverted
181                 int fb = bx.get(i)
182                 if fb >= minBar and (na(zbar) or fb > zbar)
183                     zt := top.get(i)
184                     zb := bot.get(i)
185                     zbar := fb
186                 top.remove(i)
187                 bot.remove(i)
188                 bx.remove(i)
189     [zt, zb, zbar]
190
191 // CISD reference: open of the earliest candle in the consecutive opposing series that
192 // delivered price into the current extreme (bull: down-close candles, bear: up-close).
193 cisdLevel(int d) =>
194     float lvl = na
195     int lb = na
196     bool run = false
197     for i = 0 to cisdScan
198         bool opp = d == 1 ? close[i] < open[i] : close[i] > open[i]
199         if opp
200             lvl := open[i]
201             lb := bar_index - i
202             run := true

```

```

203         else if run
204             break
205         [lvl, lb]
206
207 // Draw on liquidity: nearest two untouched pools in trade direction, at least minDist away.
208 findTargets(int d, float entry, float minDist) =>
209     array<float> pools = d == 1 ? bslPx : sslPx
210     float t1 = na
211     float t2 = na
212     for p in pools
213         if d * (p - entry) >= minDist
214             if na(t1) or d * (t1 - p) > 0
215                 t2 := t1
216                 t1 := p
217             else if p != t1 and (na(t2) or d * (t2 - p) > 0)
218                 t2 := p
219     [t1, t2]
220
221 // ----- Setup state machine -----
222 // st: 0 idle · 1 swept (waiting V + CISD + IFVG) · 2 IFVG armed (waiting retest) · 3 in trade
223 type Setup
224     int dir
225     int st = 0
226     float lvl
227     int lvlBar
228     int sweepBar
229     float ext
230     int extBar
231     bool vDone = false
232     int vBar
233     float vPx
234     float cisd
235     int cisdBar
236     bool cisdDone = false
237     int cisdConfBar
238     bool zDone = false
239     float zTop
240     float zBot
241     int zBar
242     int armBar
243     float far // V-leg extreme in trade direction (for discount / premium)
244     float entry
245     float sl
246     float tp1
247     float tp2
248     float risk
249     float r1
250     float r2
251     bool tp1Hit = false
252     int entryBar
253     box zBox
254     line dSweepLn
255     label dSweepLb
256     label dVLb
257     line dCisdLn
258     label dCisdLb
259     box bxRisk
260     box bxRew
261     line lT1
262     label bS
263     label bT1
264     label bT2
265
266 method reset(Setup s) =>
267     s.st := 0
268     s.vDone := false
269     s.cisdDone := false
270     s.zDone := false
271     s.tp1Hit := false
272     s.zBox := na
273     s.dSweepLn := na
274     s.dSweepLb := na
275     s.dVLb := na
276     s.dCisdLn := na

```

```

277     s.dCisdLb := na
278     s.bxRisk  := na
279     s.bxRew   := na
280     s.lT1     := na
281     s.bS      := na
282     s.bT1     := na
283     s.bT2     := na
284     true
285
286 method start(Setup s, float lvl, int lvlBar) =>
287     float adv = s.dir == 1 ? low : high
288     if s.st == 0
289         [c, cb] = cisdLevel(s.dir)
290         s.st      := 1
291         s.lvl     := lvl
292         s.lvlBar  := lvlBar
293         s.sweepBar := bar_index
294         s.ext     := adv
295         s.extBar  := bar_index
296         s.cisd    := c
297         s.cisdBar := cb
298         s.far     := s.dir == 1 ? high : low
299     else if s.st == 1 and s.dir * (s.lvl - lvl) > 0
300         s.lvl     := lvl
301         s.lvlBar  := lvlBar
302     true
303
304 // "+1.5R" / "-1R"
305 fmtR(float r) => (r >= 0 ? "+" : "") + str.toString(r, "#.##") + "R"
306
307 method setTradeX(Setup s, int x) =>
308     box.set_right(s.bxRisk, x)
309     box.set_right(s.bxRew, x)
310     line.set_x2(s.lT1, x)
311     label.set_x(s.bS, x)
312     label.set_x(s.bT1, x)
313     label.set_x(s.bT2, x)
314     true
315
316 // bias: HTF bias agrees with this direction · allow: killzone, daily limits and no open opposite trade
317 method update(Setup s, float atrV, float minorLvl, float zt, float zb, int zbar, bool bias, bool allow)
=>
318     int d = s.dir
319     color c = d == 1 ? bullCol : bearCol
320     float adv = d == 1 ? low : high
321     float fav = d == 1 ? high : low
322     bool evArm = false
323     bool evEntry = false
324     bool evTp1 = false
325     bool evTp2 = false
326     bool evSl = false
327     bool evBe = false
328     float resR = na // realised R when the trade closes on this bar
329     float part = partialPct / 100
330
331     if s.st == 1
332         // New extreme → any partial confirmation is void, rebuild from the new low/high
333         if d * (s.ext - adv) > 0
334             [cl, cb] = cisdLevel(d)
335             s.ext     := adv
336             s.extBar  := bar_index
337             s.vDone   := false
338             s.cisdDone := false
339             s.zDone   := false
340             s.cisd    := cl
341             s.cisdBar := cb
342             s.far     := fav
343             s.far := na(s.far) or d * (fav - s.far) > 0 ? fav : s.far
344             if not na(zbar)
345                 s.zTop := zt
346                 s.zBot := zb
347                 s.zBar := zbar
348                 s.zDone := true

```

```

349     if not s.vDone and bar_index - s.extBar <= vMaxBars and d * (fav - s.ext) >= vAtrMult * atrV and
d * (close - s.lvl) > 0
350         s.vDone := true
351         s.vBar := bar_index
352         s.vPx := fav
353     if not s.cisdDone and not na(s.cisd) and d * (close - s.cisd) > 0
354         s.cisdDone := true
355         s.cisdConfBar := bar_index
356
357     if s.vDone and s.cisdDone and s.zDone
358         s.st := 2
359         s.armBar := bar_index
360         evArm := true
361         color cf = color.new(c, 20)
362         s.dSweepLn := line.new(s.lvlBar, s.lvl, s.sweepBar, s.lvl, color = c, style =
line.style_dashed)
363         s.dSweepLb := label.new(s.extBar, s.ext, "1 Sweep", color = cf, textcolor = color.white,
364             style = d == 1 ? label.style_label_up : label.style_label_down, size = lblSz,
365             tooltip = (d == 1 ? "Sell-side" : "Buy-side") + " liquidity at " + str.toString(s.lvl,
format.mintick) + " taken and reclaimed")
366         s.dVLb := label.new(s.vBar, s.vPx, "2 V", color = cf, textcolor = color.white,
367             style = d == 1 ? label.style_label_down : label.style_label_up, size = lblSz,
368             tooltip = "V-shape recovery: fast displacement away from the sweep")
369         s.dCisdLn := line.new(s.cisdBar, s.cisd, s.cisdConfBar, s.cisd, color = c, width = 2)
370         s.dCisdLb := label.new(s.cisdBar, s.cisd, "3 CISD", color = cf, textcolor = color.white,
371             style = label.style_label_right, size = lblSz,
372             tooltip = "Change in state of delivery: close through " + str.toString(s.cisd,
format.mintick))
373         s.zBox := box.new(s.zBar, math.max(s.zTop, s.zBot), bar_index, math.min(s.zTop, s.zBot),
374             border_color = c, bgcolor = color.new(c, 82), text = "4 IFVG", text_color = c,
375             text_size = lblSz, text_halign = text.align_left, text_valign = text.align_center)
376     else if bar_index - s.sweepBar > alignBars or (not s.vDone and bar_index - s.extBar > vMaxBars)
377         s.reset()
378
379     else if s.st == 2
380         float zNear = d == 1 ? s.zTop : s.zBot
381         float zFar = d == 1 ? s.zBot : s.zTop
382         box.set_right(s.zBox, bar_index)
383         s.far := d * (fav - s.far) > 0 ? fav : s.far
384         if bar_index > s.armBar
385             bool broke = d * (close - zFar) < 0 or d * (adv - s.ext) < 0
386             bool expired = bar_index - s.armBar > retestMax
387             bool touched = d * (adv - zNear) <= 0
388             bool sweep2 = d * (adv - minorLvl) < 0 and d * (close - minorLvl) > 0
389             bool holds = d * (close - zFar) >= 0
390             if broke or expired
391                 if showFailed
392                     box.set_bgcolor(s.zBox, color.new(color.gray, 88))
393                     box.set_border_color(s.zBox, color.gray)
394                     box.set_text_color(s.zBox, color.gray)
395                     box.set_text(s.zBox, "IFVG cancelled")
396                 else
397                     box.delete(s.zBox)
398                     line.delete(s.dSweepLn)
399                     label.delete(s.dSweepLb)
400                     label.delete(s.dVLb)
401                     line.delete(s.dCisdLn)
402                     label.delete(s.dCisdLb)
403                 s.reset()
404             else if touched and sweep2 and holds and bias and allow
405                 float entry = close
406                 float legSz = math.abs(s.far - s.ext)
407                 float depth = legSz > 0 ? d * (s.far - entry) / legSz : 0.0
408                 float risk = math.max(math.abs(entry - (adv - d * slBuf * atrV)), minSlAtr * atrV)
409                 [dol, dol2] = findTargets(d, entry, rrl * risk)
410                 bool okSize = risk <= maxSlAtr * atrV
411                 bool okDol = not requireDol or not na(dol)
412                 bool okPd = depth >= pdMin
413                 // A retest that fails the stop-size / DOL / discount filters is skipped; the zone stays
armed.
414                 if okSize and okDol and okPd
415                     s.st := 3
416                     s.entryBar := bar_index
417                     s.entry := entry

```

```

418         s.risk      := risk
419         s.sl        := entry - d * risk
420         float cap1 = entry + d * rr1 * risk
421         float cap2 = entry + d * rr2 * risk
422         if tpMode == "Fixed R:R"
423             s.tp1 := cap1
424             s.tp2 := cap2
425         else
426             [t1, t2] = findTargets(d, entry, 0.5 * risk)
427             s.tp1 := na(t1) or d * (t1 - cap1) > 0 ? cap1 : t1
428             s.tp2 := na(t2) or d * (t2 - cap2) > 0 ? cap2 : t2
429             if d * (s.tp2 - s.tp1) <= 0
430                 s.tp2 := entry + d * math.max(rr2 * risk, math.abs(s.tp1 - entry) + 0.5 *
risk)
431
432             s.r1 := math.abs(s.tp1 - entry) / risk
433             s.r2 := math.abs(s.tp2 - entry) / risk
434             evEntry := true
435
436             int x2 = bar_index + 5
437             string tip = "Entry " + str.toString(s.entry, format.mintick) +
438                 "\nSL " + str.toString(s.sl, format.mintick) +
439                 "\nTP1 " + str.toString(s.tp1, format.mintick) + " (" + str.toString(s.r1,
440 "#.##") + "R)" +
441                 "\nTP2 " + str.toString(s.tp2, format.mintick) + " (" + str.toString(s.r2,
442 "#.##") + "R)" +
443                 "\nDraw on liquidity " + (na(dol) ? "none beyond TP1" : str.toString(dol,
format.mintick)) +
444                 "\nRetracement " + str.toString(depth * 100, "#") + "% of V leg"
445             label.new(bar_index, adv, d == 1 ? "5 BUY" : "5 SELL", color = c, textcolor =
color.white,
446                 style = d == 1 ? label.style_label_up : label.style_label_down, size = lblSz,
tooltip = tip)
447             box.set_border_width(s.zBox, 2)
448             s.bxBRisk := box.new(bar_index, math.max(s.entry, s.sl), x2, math.min(s.entry, s.sl),
449                 border_color = color.new(slCol, 50), bgcolor = color.new(slCol, 80))
450             s.bxBReW := box.new(bar_index, math.max(s.entry, s.tp2), x2, math.min(s.entry,
s.tp2),
451                 border_color = color.new(tgtCol, 50), bgcolor = color.new(tgtCol, 80))
452             s.lT1 := line.new(bar_index, s.tp1, x2, s.tp1, color = tgtCol, style =
line.style_dashed)
453             s.bS := label.new(x2, s.sl, "SL " + str.toString(s.sl, format.mintick), color =
color(na),
454                 textcolor = slCol, style = label.style_label_left, size = lblSz)
455             s.bT1 := label.new(x2, s.tp1, "TP1 " + str.toString(s.r1, "#.##") + "R", color =
color(na),
456                 textcolor = tgtCol, style = label.style_label_left, size = lblSz)
457             s.bT2 := label.new(x2, s.tp2, "TP2 " + str.toString(s.r2, "#.##") + "R", color =
color(na),
458                 textcolor = tgtCol, style = label.style_label_left, size = lblSz)
459
460         else if s.st == 3
461             if bar_index > s.entryBar
462                 s.setTradeX(bar_index + 5)
463                 bool hitSl = d * (adv - s.sl) <= 0
464                 bool hitTp1 = d * (fav - s.tp1) >= 0
465                 bool hitTp2 = d * (fav - s.tp2) >= 0
466                 // Conservative: if SL and a target print on the same bar, the stop is assumed first
467                 if hitSl
468                     if s.tp1Hit
469                         evBe := true
470                         resR := part * s.r1 + (1 - part) * d * (s.sl - s.entry) / s.risk
471                     else
472                         evSl := true
473                         resR := -1.0
474                     label.new(bar_index, adv, (s.tp1Hit ? "BE " : "x SL ") + fmtR(resR), color = s.tp1Hit ?
color.gray : slCol,
475                         textcolor = color.white, style = d == 1 ? label.style_label_up :
label.style_label_down, size = lblSz)
476                 s.setTradeX(bar_index)
477                 s.reset()
478             else if hitTp2
479                 if not s.tp1Hit
480                     evTp1 := true
481                 evTp2 := true

```

```

479         resR := part * s.r1 + (1 - part) * s.r2
480         label.new(bar_index, s.tp2, "✓ TP2 " + fmtR(resR), color = tgtCol, textcolor =
color.white,
481                 style = d == 1 ? label.style_label_down : label.style_label_up, size = lblSz)
482         s.setTradeX(bar_index)
483         s.reset()
484     else
485         if hitTp1 and not s.tp1Hit
486             s.tp1Hit := true
487             evTp1 := true
488             label.new(bar_index, s.tp1, "✓ TP1", color = color.new(tgtCol, 30), textcolor =
color.white,
489                     style = d == 1 ? label.style_label_down : label.style_label_up, size = lblSz)
490             if beAfterTp1
491                 s.sl := s.entry
492                 box.set_bgcolor(s.bxRisk, color.new(color.gray, 90))
493                 box.set_border_color(s.bxRisk, color.new(color.gray, 70))
494                 label.set_y(s.bS, s.entry)
495                 label.set_text(s.bS, "SL moved to entry")
496             if bar_index - s.entryBar >= tradeMax
497                 float openR = d * (close - s.entry) / s.risk
498                 resR := s.tp1Hit ? part * s.r1 + (1 - part) * openR : openR
499                 label.new(bar_index, close, "Time exit " + fmtR(resR), color = color.gray, textcolor
= color.white,
500                         style = label.style_label_left, size = lblSz)
501                 s.setTradeX(bar_index)
502                 s.reset()
503
504         [evArm, evEntry, evTp1, evTp2, evSl, evBe, resR]
505
506 // ----- Engine -----
507 var Setup bull = Setup.new(1)
508 var Setup bear = Setup.new(-1)
509
510 // 1 · register new liquidity pools
511 if not na(ph)
512     addPool(bslPx, bslBar, bslLn, ph, bar_index - swingLen)
513 if not na(pl)
514     addPool(sslPx, sslBar, sslLn, pl, bar_index - swingLen)
515
516 // previous-day high / low become liquidity pools at the start of each new day
517 var float dH = na
518 var float dL = na
519 var int dHB = na
520 var int dLB = na
521 if timeframe.change("D")
522     if usePdl and not na(dH)
523         addPool(bslPx, bslBar, bslLn, dH, dHB)
524         addPool(sslPx, sslBar, sslLn, dL, dLB)
525     dH := high
526     dHB := bar_index
527     dL := low
528     dLB := bar_index
529 else
530     if na(dH) or high > dH
531         dH := high
532         dHB := bar_index
533     if na(dL) or low < dL
534         dL := low
535         dLB := bar_index
536
537 // Asia session high / low become liquidity pools when the session ends
538 bool inAsia = not na(time(timeframe.period, asiaSess, kzTz))
539 var float aH = na
540 var float aL = na
541 var int aHB = na
542 var int aLB = na
543 if inAsia
544     if not inAsia[1] or na(aH)
545         aH := high
546         aHB := bar_index
547         aL := low
548         aLB := bar_index
549     else

```

```

550         if high > aH
551             aH := high
552             aHB := bar_index
553         if low < aL
554             aL := low
555             aLB := bar_index
556     else if inAsia[1] and useAsia and not na(aH)
557         addPool(bslPx, bslBar, bslLn, aH, aHB)
558         addPool(sslPx, sslBar, sslLn, aL, aLB)
559
560 // daily trade / loss limits (reset with the trading day)
561 var int dayTrades = 0
562 var int dayLosses = 0
563 if timeframe.change("D")
564     dayTrades := 0
565     dayLosses := 0
566 bool dayOk = dayTrades < maxDay and dayLosses < maxLossDay
567
568 // 2 · liquidity sweeps start (or deepen) a setup
569 [sslLv1, sslB] = scanSweeps(sslPx, sslBar, sslLn, 1)
570 [bslLv1, bslB] = scanSweeps(bslPx, bslBar, bslLn, -1)
571 if showBull and bullBias and not na(sslLv1)
572     bull.start(sslLv1, sslB)
573 if showBear and bearBias and not na(bslLv1)
574     bear.start(bslLv1, bslB)
575
576 // 3 · FVG inversions (only FVGs formed around the sweep, inverted by a displacement candle)
577 int bullMin = bull.st == 1 and bullDisp ? bull.sweepBar - fvgLook : bar_index + 1
578 int bearMin = bear.st == 1 and bearDisp ? bear.sweepBar - fvgLook : bar_index + 1
579 [bzt, bzb, bzbar] = scanInversions(bearFvgTop, bearFvgBot, bearFvgBar, 1, bullMin)
580 [szt, szb, szbar] = scanInversions(bullFvgTop, bullFvgBot, bullFvgBar, -1, bearMin)
581
582 if high < low[2] and low[2] - high >= fvgMinAtr * atr
583     addFvg(bearFvgTop, bearFvgBot, bearFvgBar, low[2], high, bar_index - 1)
584 if low > high[2] and low - high[2] >= fvgMinAtr * atr
585     addFvg(bullFvgTop, bullFvgBot, bullFvgBar, low, high[2], bar_index - 1)
586
587 // 4 · advance state machines
588 // one position at a time: a direction may not enter while the other is in a trade
589 [bArm, bEntry, bTp1, bTp2, bSl, bBe, bR] = bull.update(atr, minorLow, bzt, bzb, bzbar, bullBias, inKz and
dayOk and bear.st != 3)
590 [sArm, sEntry, sTp1, sTp2, sSl, sBe, sR] = bear.update(atr, minorHigh, szt, szb, szbar, bearBias, inKz
and dayOk and bull.st != 3)
591 dayTrades += (bEntry ? 1 : 0) + (sEntry ? 1 : 0)
592 dayLosses += (not na(bR) and bR < 0 ? 1 : 0) + (not na(sR) and sR < 0 ? 1 : 0)
593
594 // ----- Stats -----
595 var int nTrades = 0
596 var int nClosed = 0
597 var int nWin = 0
598 var int nTp1 = 0
599 var int nTp2 = 0
600 var int nLoss = 0
601 var float netR = 0.0
602 nTrades += (bEntry ? 1 : 0) + (sEntry ? 1 : 0)
603 nTp1 += (bTp1 ? 1 : 0) + (sTp1 ? 1 : 0)
604 nTp2 += (bTp2 ? 1 : 0) + (sTp2 ? 1 : 0)
605 nLoss += (bSl ? 1 : 0) + (sSl ? 1 : 0)
606 for r in array.from(bR, sR)
607     if not na(r)
608         nClosed += 1
609         nWin += r > 0 ? 1 : 0
610         netR += r
611
612 statRow(table t, int row, string k, string v, color c) =>
613     table.cell(t, 0, row, k, text_color = color.silver, text_size = size.small, text_halign =
text.align_left)
614     table.cell(t, 1, row, v, text_color = c, text_size = size.small, text_halign = text.align_right)
615
616 pct(int a, int b) => b > 0 ? str.toString(100.0 * a / b, "#") + "%" : "-"
617
618 stateName(Setup s) =>
619     s.st == 0 ? "Idle" : s.st == 1 ? "Swept – waiting V/CISD/IFVG" : s.st == 2 ? "IFVG armed – waiting
retest" : "In trade"

```

```

620
621 string tPos = tablePos == "Bottom right" ? position.bottom_right : tablePos == "Top left" ?
    position.top_left :
622     tablePos == "Bottom left" ? position.bottom_left : position.top_right
623 var table stats = table.new(tPos, 2, 13, bgcolor = color.new(#131722, 10), border_width = 1)
624 if showStats and barstate.islast
625     table.cell(stats, 0, 0, "IFVG Sniper · " + timeframe.period + (timeframe.period == "15" ? "" : "
    (tuned for 15)"),
626     text_color = color.white, text_size = size.small)
627     table.merge_cells(stats, 0, 0, 1, 0)
628     statRow(stats, 1, "Trades closed", str.toString(nClosed) + " / " + str.toString(nTrades),
    color.white)
629     statRow(stats, 2, "Win rate (R > 0)", pct(nWin, nClosed), tgtCol)
630     statRow(stats, 3, "TP1 reached", pct(nTp1, nClosed), tgtCol)
631     statRow(stats, 4, "TP2 reached", pct(nTp2, nClosed), tgtCol)
632     statRow(stats, 5, "Stopped before TP1", pct(nLoss, nClosed), slCol)
633     statRow(stats, 6, "Net R", str.toString(netR, "#.##"), netR >= 0 ? tgtCol : slCol)
634     statRow(stats, 7, "Avg R / trade", nClosed > 0 ? str.toString(netR / nClosed, "#.##") : "-", netR >=
    0 ? tgtCol : slCol)
635     statRow(stats, 8, "HTF bias", not useHtf ? "off" : bullBias ? "Bullish" : bearBias ? "Bearish" :
    "Neutral – no trades",
636     not useHtf ? color.gray : bullBias ? bullCol : bearBias ? bearCol : color.gray)
637     statRow(stats, 9, "Killzone", not useKz ? "off" : inKz ? "Open" : "Closed", inKz ? tgtCol :
    color.gray)
638     statRow(stats, 10, "Today", str.toString(dayTrades) + " trades · " + str.toString(dayLosses) + "
    losses" + (dayOk ? "" : " – done"),
639     dayOk ? color.white : slCol)
640     statRow(stats, 11, "Bull", stateName(bull), bullCol)
641     statRow(stats, 12, "Bear", stateName(bear), bearCol)
642
643 // ----- Data window -----
644 plot(bull.st == 3 ? bull.entry : na, "Bull entry", bullCol, display = display.data_window)
645 plot(bull.st == 3 ? bull.sl : na, "Bull SL", slCol, display = display.data_window)
646 plot(bull.st == 3 ? bull.tp1 : na, "Bull TP1", tgtCol, display = display.data_window)
647 plot(bull.st == 3 ? bull.tp2 : na, "Bull TP2", tgtCol, display = display.data_window)
648 plot(bear.st == 3 ? bear.entry : na, "Bear entry", bearCol, display = display.data_window)
649 plot(bear.st == 3 ? bear.sl : na, "Bear SL", slCol, display = display.data_window)
650 plot(bear.st == 3 ? bear.tp1 : na, "Bear TP1", tgtCol, display = display.data_window)
651 plot(bear.st == 3 ? bear.tp2 : na, "Bear TP2", tgtCol, display = display.data_window)
652
653 // ----- Alerts -----
654 alertcondition(bArm, "Bull IFVG armed", "IFVG Sniper: bullish sweep + V + CISD + IFVG armed on
    {{ticker}} ({{interval}})")
655 alertcondition(bEntry, "Bull entry", "IFVG Sniper: BUY on {{ticker}} @ {{close}} – IFVG retest +
    sweep")
656 alertcondition(bTp1, "Bull TP1", "IFVG Sniper: bullish TP1 hit on {{ticker}}")
657 alertcondition(bTp2, "Bull TP2", "IFVG Sniper: bullish TP2 hit on {{ticker}}")
658 alertcondition(bSl, "Bull stop", "IFVG Sniper: bullish stop hit on {{ticker}}")
659 alertcondition(sArm, "Bear IFVG armed", "IFVG Sniper: bearish sweep + V + CISD + IFVG armed on
    {{ticker}} ({{interval}})")
660 alertcondition(sEntry, "Bear entry", "IFVG Sniper: SELL on {{ticker}} @ {{close}} – IFVG retest +
    sweep")
661 alertcondition(sTp1, "Bear TP1", "IFVG Sniper: bearish TP1 hit on {{ticker}}")
662 alertcondition(sTp2, "Bear TP2", "IFVG Sniper: bearish TP2 hit on {{ticker}}")
663 alertcondition(sSl, "Bear stop", "IFVG Sniper: bearish stop hit on {{ticker}}")
664
665 if bEntry
666     alert("IFVG Sniper BUY " + syminfo.ticker + " | entry " + str.toString(bull.entry, format.mintick) +
667     " | SL " + str.toString(bull.sl, format.mintick) + " | TP1 " + str.toString(bull.tp1,
    format.mintick) +
668     " | TP2 " + str.toString(bull.tp2, format.mintick), alert.freq_once_per_bar_close)
669 if sEntry
670     alert("IFVG Sniper SELL " + syminfo.ticker + " | entry " + str.toString(bear.entry, format.mintick) +
671     " | SL " + str.toString(bear.sl, format.mintick) + " | TP1 " + str.toString(bear.tp1,
    format.mintick) +
672     " | TP2 " + str.toString(bear.tp2, format.mintick), alert.freq_once_per_bar_close)
673

```